

Process API

CS 232 Operating Systems
Fall 2019

What is an API?

- Application Programming Interface
- Every OS provides a set of functions called System Calls
- System calls are
 - OS code
 - Runs at a higher “privilege level”
 - Can do things user code can’t i.e. access hardware
- System calls for each OS may be different

POSIX

- POSIX (Portable OS Interface) Standard
 - API Standard
 - Ensures compatibility across OSs
 - Programs using POSIX API sure to run on POSIX-compliant OS
 - Most OS provide some sort of POSIX compliance
- Libraries provide an easy-to-use interface to System Calls
 - C language has libc library
 - `printf()` calls the `write()` system call

Linux System Calls

- Linux kernel provides a set of System Calls
- Among them for process management:
 - `fork ()` - creates a process
 - all processes get created via a `fork()`
 - The `init` process is the first process in linux
 - `exec ()` - executes a given executable
 - `exit ()` - terminates a process
 - `wait ()` - parent can wait for a child to terminate

Process creation

- A new process is created using the `fork()` system call
- An exact copy of the parent process
- If successful, the `fork()` system call returns twice:
 - returns the value 0 in child
 - returns the value child's PID in parent
- Both parent and child will resume execution from after `fork()` (can't assume which one will run first)
- Both will have their own address spaces

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4
5  int main(int argc, char *argv[]) {
6      printf("hello world (pid:%d)\n", (int) getpid());
7      int rc = fork();
8      if (rc < 0) {                // fork failed; exit
9          fprintf(stderr, "fork failed\n");
10         exit(1);
11     } else if (rc == 0) { // child (new process)
12         printf("hello, I am child (pid:%d)\n", (int) getpid());
13     } else {                    // parent goes down this path (main)
14         printf("hello, I am parent of %d (pid:%d)\n",
15             rc, (int) getpid());
16     }
17     return 0;
18 }
19

```

Figure 5.1: Calling **fork ()** (**p1.c**)

When you run this program (called **p1.c**), you'll see the following:

```

prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>

```

Process termination

- A process terminates either:
 - voluntarily, by calling the `exit()` system call
 - or by the operating system
- Parent could call `wait()` syscall to wait for child to terminate
 - `wait()` could be blocking or not
 - zombie: a process finished execution but hasn't been reaped by parent yet
- If parent terminates first, child is orphan
 - `init` process is the parent of all orphans

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main(int argc, char *argv[]) {
7      printf("hello world (pid:%d)\n", (int) getpid());
8      int rc = fork();
9      if (rc < 0) {          // fork failed; exit
10         fprintf(stderr, "fork failed\n");
11         exit(1);
12     } else if (rc == 0) { // child (new process)
13         printf("hello, I am child (pid:%d)\n", (int) getpid());
14     } else {               // parent goes down this path (main)
15         int rc_wait = wait(NULL);
16         printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
17             rc, rc_wait, (int) getpid());
18     }
19     return 0;
20 }
21
```

Figure 5.2: Calling `fork()` And `wait()` (`p2.c`)

Running another process

- `fork()` created a copy of parent process
- `exec()` overwrites the process memory image with another program
 - it never returns
 - this way child can run a different program than parent
 - `exec()` has different variants with different arguments

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <sys/wait.h>
6
7  int main(int argc, char *argv[]) {
8      printf("hello world (pid:%d)\n", (int) getpid());
9      int rc = fork();
10     if (rc < 0) {                // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child (new process)
14         printf("hello, I am child (pid:%d)\n", (int) getpid());
15         char *myargs[3];
16         myargs[0] = strdup("wc"); // program: "wc" (word count)
17         myargs[1] = strdup("p3.c"); // argument: file to count
18         myargs[2] = NULL;          // marks end of array
19         execvp(myargs[0], myargs); // runs word count
20         printf("this shouldn't print out");
21     } else {                      // parent goes down this path (main)
22         int rc_wait = wait(NULL);
23         printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
24               rc, rc_wait, (int) getpid());
25     }
26     return 0;
27 }
28

```

The Shell

- Linux shell is a user program
- Each time you run a command it'd:
 - call `fork()` and create a child in its own image
 - that child would call `exec()` with your command as its argument
 - the shell would `wait()` for the child to finish
 - after having finished the child is reaped by the shell
 - the shell displays the prompt to accept a new command
- (almost) All Linux commands are executable programs
- This fork/exec scheme lets the shell set up the child's environment before it starts running
 - By default the child would inherit the environment of its parent

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <string.h>
5  #include <fcntl.h>
6  #include <sys/wait.h>
7
8  int main(int argc, char *argv[]) {
9      int rc = fork();
10     if (rc < 0) {                // fork failed; exit
11         fprintf(stderr, "fork failed\n");
12         exit(1);
13     } else if (rc == 0) { // child: redirect standard output to a file
14         close(STDOUT_FILENO);
15         open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
16
17         // now exec "wc"...
18         char *myargs[3];
19         myargs[0] = strdup("wc"); // program: "wc" (word count)
20         myargs[1] = strdup("p4.c"); // argument: file to count
21         myargs[2] = NULL;          // marks end of array
22         execvp(myargs[0], myargs); // runs word count
23     } else {                      // parent goes down this path (main)
24         int rc_wait = wait(NULL);
25     }
26     return 0;
27 }

```

Pipes

- The Shell pipes (|) work via redirection
 - When the shell creates the two processes
 - The output of the first process is connected to input of kernel pipe (a queue)
 - The input of the second process is connected to the output of kernel pipe
 - Thus one process writes to the kernel pipe, the other reads from it